



MICROSERVICES COURSE

Session 21

Service Mesh & mTLS: Istio

Sidecar Proxy Pattern · Mutual TLS · Traffic Management · Mesh-Level Resilience



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Wednesday

TIME

3:00–5:30 PM

DURATION

2.5 Hours

PHASE

Phase 3 · Advanced & Enterprise

Where We Are



Session 21 builds on Sessions 19–20 (application-level security) — today adds infrastructure-level security and traffic control via a service mesh. Full 29-session roadmap lives in the Kickoff deck only.

Closing the Loop from Sessions 19–20



S19–S20

Application-Level Security

- JWT validated by Spring Security
- OAuth2 Resource Server config
- Proves WHICH USER is calling



S21 — TODAY

Infrastructure-Level Security

- mTLS between every pod, automatically
- No application code changes needed
- Proves WHICH SERVICE is calling

Together: Defense in Depth

Two independent layers. If one fails or is misconfigured, the other still protects the platform.

Repeating Cross-Cutting Concerns in Every Service



Resilience4j dependencies and annotations copied into Order, Payment, Inventory (S4-5)



OAuth2 Resource Server config duplicated across every service that needs it (S20)



Micrometer Tracing dependencies added service by service (S17)



Worse: traffic between Order Service and Inventory Service is currently PLAIN HTTP — anyone on the cluster network can read it

The Gap: Service-to-Service Traffic Is Plaintext

We secured the EDGE (Gateway, Sessions 3 & 20) — but not what happens AFTER a request passes through it.

Order Service — *plain HTTP, unencrypted* —> Inventory Service

Anyone who can sniff traffic on the Kubernetes network can read every request between our services.

The question this session answers:

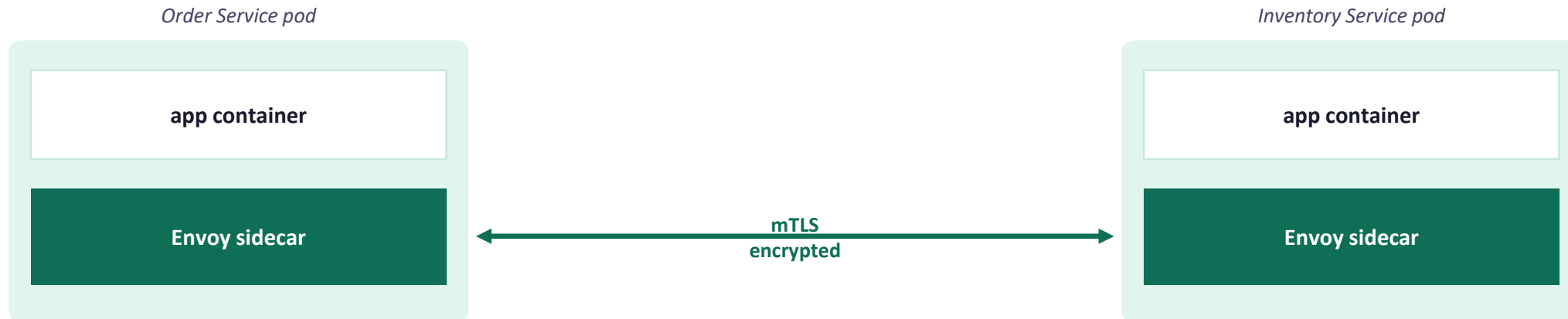
What if encryption, retries, circuit breaking, and telemetry could live in ONE place — applied uniformly, without touching a single pom.xml?

One Component Intercepting Traffic — At Every Pod

WITHOUT Istio



WITH Istio — a sidecar (Envoy) is injected into EVERY pod



Istiod (Control Plane) configures every Envoy sidecar's routing, security, and telemetry rules

Data Plane vs Control Plane

Envoy Sidecar — Data Plane

Role

Intercepts and handles every byte of network traffic for its pod

Analogy

Like Session 2's API Gateway — but one per pod, not one per platform

Istiod — Control Plane

Role

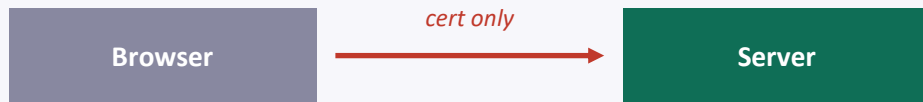
Distributes configuration (routing, security, telemetry rules) to every sidecar

Analogy

Like Session 1's Config Server — but for mesh behavior, not application properties

One-Way TLS vs Mutual TLS

One-Way TLS (e.g. browsing https://...)



Only the SERVER proves its identity. The browser does not prove its own — anyone's browser can connect.

Mutual TLS — Istio between mesh services



BOTH sides present and verify certificates. Neither can be impersonated by an attacker with network access.

mTLS Proves WHICH SERVICE — Not WHICH USER



mTLS says: "I really am order-service."

It says nothing about which USER that call is acting for — that's still the OAuth2 JWT's job from Session 19-20.

-
- mTLS (Session 21): authenticates the CALLING SERVICE's identity — WHICH service is connecting
 - OAuth2 JWT (Sessions 19-20): authenticates the USER on whose behalf the call is made
 - The two are complementary layers, not competitors — neither replaces the other

Install Istio + Enable Sidecar Injection

```
# Install Istio into the Session 15-16 Kubernetes cluster
istioctl install --set profile=demo -y

# Label the 'ecommerce' namespace for automatic sidecar injection
kubectl label namespace ecommerce istio-injection=enabled

# Restart existing deployments so the sidecar gets injected
kubectl rollout restart deployment -n ecommerce

# Verify: each pod now shows 2/2 containers ready (app + istio-proxy)
kubectl get pods -n ecommerce
```

Sidecar injection only applies to NEW pod creation — that's why we restart the deployments.

Enforce STRICT mTLS Mesh-Wide

```
# peer-authentication.yaml — enforce STRICT mTLS mesh-wide
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ecommerce
spec:
  mtls:
    mode: STRICT    # reject any plaintext traffic entirely

kubectl apply -f peer-authentication.yaml

# Verify mTLS is active for all traffic to product-service:
istioctl authn tls-check product-service.ecommerce.svc.cluster.local
# Expected: STATUS = OK
```

Plaintext Before → Encrypted After — Zero App Code Changes

BEFORE — no PeerAuthentication

```
tcpdump capture:  
  
GET /api/v1/inventory/check  
Host: inventory-service  
Authorization: Bearer eyJhbG...
```

Anyone on the network can read this.

AFTER — STRICT mTLS applied

```
tcpdump capture:  
  
0x4a2f 0x9b1e 0xc7d3 0x8a44  
0xf120 0x33ab 0x77c9 0x0e5f  
(unreadable cipher text)
```

Fully encrypted — zero changes to product-service or order-service code.

VirtualService vs DestinationRule



VirtualService

Defines ROUTING rules — which requests go where, and in what proportion.

e.g. 80% of traffic to 'stable', 20% to 'canary'



DestinationRule

Defines POLICY once traffic reaches a destination — load balancing, connection pooling, outlier detection.

e.g. eject a pod after 5 consecutive 5xx errors

Together they replace Session 14's replica-ratio Canary approximation with NATIVE, precise, percentage-based traffic splitting.

Weighted Traffic Split — 80/20 (Native, Not Approximated)

```
# virtual-service.yaml — precise split, replacing S14's replica-count approx
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: product-service
  namespace: ecommerce
spec:
  hosts: [product-service]
  http:
    - route:
        - destination: {host: product-service, subset: stable}
          weight: 80
        - destination: {host: product-service, subset: canary}
          weight: 20
```

DestinationRule — Mesh-Level Circuit Breaker

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: product-service
spec:
  host: product-service
  subsets:
    - name: stable
      labels: {version: stable}
    - name: canary
      labels: {version: canary}
  trafficPolicy:
    outlierDetection:      # <- mesh-level Circuit Breaker
      consecutive5xxErrors: 5
      baseEjectionTime: 30s # eject unhealthy pod for 30s
```

outlierDetection Works With ZERO Java Code



The DestinationRule you just saw ejects an unhealthy pod from load balancing — entirely inside the Envoy sidecar, configured declaratively via YAML.

Resilience4j @CircuitBreaker (Session 4)

- Lives inside application code
- Requires spring-boot-starter-aop
- Can build a domain-aware fallback (e.g. OrderResponse PENDING)

Istio outlierDetection (Today)

- Lives in the sidecar, outside any app code
- No dependency, no annotation, no restart
- Can only reject/eject a connection — no domain response

Service Mesh vs Application-Level Patterns

Need mTLS without code changes?

→ **Service Mesh (Istio)**

Traffic management (canary / circuit break)?

→ **Service Mesh**

Team owns and operates the cluster?

→ **Service Mesh**

Simple deployment, no K8s expertise available?

→ **Application patterns (Resilience4j)**

Sidecar resource overhead unacceptable?

→ **Application patterns**

Honest Trade-offs, Not Fairy Tales

Istio is powerful, but it is not free.

- A sidecar container in every single pod — real memory and CPU cost
 - A control plane (Istiod) that must be operated and kept healthy
 - A genuinely steep learning curve for the team running it
-

A 3-service platform, one team, one language, like early phases of THIS course's platform, may not justify a mesh at all.

A 30-service platform with five teams and three languages almost certainly does.

Know this trade-off cold for your Capstone architecture defense — "we added Istio because X" needs a real X.

Lab 17 — mTLS + Weighted Traffic Split

1

Install Istio + Enable Injection

istioctl install; label the ecommerce namespace; verify 2/2 READY pods

2

Enable STRICT mTLS

PeerAuthentication mode: STRICT; verify with istioctl authn tls-check

3

Weighted VirtualService

80/20 split (stable/canary) + DestinationRule with outlierDetection

Duration: 15 min in-session + complete as homework if needed · Builds on: S15-16 K8s, S14 Canary

Lab 17 — Acceptance Criteria

- ✓ Istio is installed and the ecommerce namespace has sidecar injection enabled
- ✓ Every pod shows 2/2 containers (app + istio-proxy)
- ✓ PeerAuthentication STRICT mode applied — istioctl authn tls-check reports OK
- ✓ VirtualService + DestinationRule configured for an 80/20 weighted split
- ✓ Repeated requests demonstrate approximately the correct traffic ratio
- ✓ outlierDetection configured on the DestinationRule

```
Commit: session-21: add-istio-mtls-and-weighted-traffic-split
```

What We Added Today

 Capability Added

Mesh-Level Security & Traffic Control

Every service-to-service call inside the cluster is now automatically encrypted and mutually authenticated — with zero application code changes.

 Services Updated

- Istio control plane installed
- product-service + order-service in mesh
- PeerAuthentication STRICT mTLS
- VirtualService + DestinationRule (80/20 + outlierDetection)

Enterprise E-Commerce Platform — After Session 21

Config+Eureka

Gateway

Resilience

K8s Pipeline

Observability

CQRS

Security

Service Mesh
NEW



Daily Quiz

8 questions · 10 minutes · Google Forms or Kahoot



Session 21 Complete

Service Mesh Active — mTLS + Native Traffic Management

Next: Session 22 — Advanced Patterns: Outbox, Idempotency & API Versioning

Monday, 3:00–5:30 PM, Online